

Exercise 3 - CASE STATEMENT

Noluthemba Madikiza

Data Analytics

Brightlearn

07 September 2026

The products

Question 1: short product catalogue. Use the price column to bucket each item into a price tier.

```
1  --Step 1: Create Catalog
2  CREATE CATALOG If NOT EXISTS electronics;
3  --Step 2: Create a Schema
4  CREATE SCHEMA If NOT EXISTS electronics.product;
5  --Step 3: Create a table
6  CREATE TABLE IF NOT EXISTS electronics.product.price (product_id INT, product_name STRING, price DECIMAL(10,2));
7  --Step 4: Insert data into the table
8  INSERT INTO electronics.product.price VALUES (1, 'Laptop', 1200), (2, 'phone', 800), (3, 'keyboard', 45.00), (4, 'Monitor', 300), (5, 'Mouse', 25.00)
9  --Step 5: check the table
10 SELECT *
11 FROM electronics.product.price;
```

Add parameter

Table	+	Q	product_id	product_name	price
1			1	Laptop	1200.00
2			2	phone	800.00
3			3	keyboard	45.00
4			4	Monitor	300.00
5			5	Mouse	25.00

Question 2:

The orders Table price price_category Customer orders with the amount paid. Label each order by its monetary tier.

▶ Run selected (1000) ✓ 1 minute ago (1s) workspace default

Generate

```
1  --Step 1: Create a Catalog
2  CREATE CATALOG IF NOT EXISTS ORDERS
3  --Step 2: Create a schema
4  CREATE SCHEMA IF NOT EXISTS ORDERS.default
5  --Step 3: Create a table
6  CREATE TABLE IF NOT EXISTS ORDERS.default.orders (order_id INT, customer_name STRING, amount DECIMAL(10,2) );
7  --Step 4: Insert data into the table
8  INSERT INTO ORDERS.default.orders VALUES
9  (1, 'Alice',150.00),
10 (2, 'Bob', 560.00),
11 (3, 'Charlie', 999.99),
12 (4, 'Diana', 45.50),
13 (5, 'Ethan',1200.00);
14 --Step 5: check the table
15 SELECT * FROM ORDERS.default.orders
```

Add parameter

Table +

	order_id	customer_name	amount
1	1	Alice	150.00
2	2	Bob	560.00
3	3	Charlie	999.99
4	4	Diana	45.50
5	5	Ethan	1200.00

▶ Run selected (1000) ✓ 4 minutes ago (14s) workspace: default

Generate

```
--Step 2: Create a schema
4 CREATE SCHEMA IF NOT EXISTS ORDERS.default
5 --Step 3: Create a table
6 CREATE TABLE IF NOT EXISTS ORDERS.default.orders (order_id INT, customer_name STRING, amount DECIMAL(10,2) );
7 --Step 4: Insert data into the table
8 INSERT INTO ORDERS.default.orders VALUES
9 (1, 'Alice',150.00),
10 (2, 'Bob', 560.00),
11 (3, 'Charlie', 999.99),
12 (4, 'Diana', 45.50),
13 (5, 'Ethan',1200.00);
14 --Step 5: check the table
15 SELECT * FROM ORDERS.default.orders
16
17 --Question 2: Customer orders with the amount paid. Label each order by its monetary tier.
18 SELECT
```

Add parameter

Table +

Q F I □ ⌂ ^ X

	customer_name	amount	order_value_category
1	Alice	150.00	budget
2	Bob	560.00	Medium Value
3	Charlie	999.99	Medium Value
4	Diana	45.50	budget
5	Ethan	1200.00	high Value

Question 3

```
6 CREATE TABLE IF NOT EXISTS employees.default.salary
7 (employee_id INT,
8 employee_name STRING,
9 department STRING,
10 salary INT
11 );
12 --Step 4: Insert data into the table
13 INSERT INTO employees.default.salary (employee_id, employee_name, department, salary)
14 VALUES
15 (1, 'John', 'IT', 85000),
16 (2, 'Sara', 'HR', 60000),
17 (3, 'Mark', 'IT', 75000),
18 (4, 'Lucy', 'Finance', 95000),
19 (5, 'Tom', 'HR', 55000);
20
21 | --Step 5: check the table
```

Id parameter

ible  

C

	¹ ₃ employee_id	^A _C employee_name	^A _C department	¹ ₃ salary
	1	John	IT	85000
	2	Sara	HR	60000
	3	Mark	IT	75000
	4	Lucy	Finance	95000
	5	Tom	HR	55000

```

19 (5, 'Tom', 'HR', 55000);
20
21 --Step 5: check the table
22 SELECT * FROM employees.default.salary
23 --Question 3: Combine the department and salary columns to label each employee with a seniority tier.
24 SELECT
25 employee_name,
26 department,
27 salary,
28 CASE
29 WHEN department = 'IT' AND salary > 80000 THEN 'senior'
30 WHEN department = 'HR' AND salary > 55000 THEN 'Experienced HR'
31 ELSE 'Staff'
32 END AS position_level
33 FROM employees.default.salary
34 DROP table if exists employees.default.salary;

```

Add parameter

Table ▼ + 🔍 🔧 📄				
	employee_name	department	salary	position_level
1	John	IT	85000	senior
2	Sara	HR	60000	Experienced HR
3	Mark	IT	75000	Staff
4	Lucy	Finance	95000	Staff
5	Tom	HR	55000	Staff

Question 4

```
1  --Step 1: Create Catalog
2  CREATE CATALOG IF NOT EXISTS students;
3  --step 2: Create schema
4  CREATE SCHEMA IF NOT EXISTS students.default;
5  --step 3: Create the table
6  CREATE TABLE IF NOT EXISTS students.default.employees (student_id INT, student_name STRING, score INT
7  );
8  --Step 4:Insert data into table
9  INSERT INTO students.default.employees VALUES (1, 'Anna', 92), (2, 'Ben', 76), (3, 'Cara',59),(4, 'David', 83), (5,
10 'Ella', 68);
10 --Step 5: check the table
11 SELECT * FROM students.default.employees;
12 |
```

Generate (Ctrl + I)

Add parameter

Table ▾ +				🔍 ⚙️ 📄 🗑️ ^ ✕			
	^{1 2} / ₃ student_id	^{A B} / _C student_name	^{1 2} / ₃ score				
1	1	Anna	92				
2	2	Ben	76				
3	3	Cara	59				
4	4	David	83				
5	5	Ella	68				

```

9  INSERT INTO students.default.employees VALUES (1, 'Anna', 92), (2, 'Ben', 76), (3, 'Cara', 59), (4, 'David', 83), ('Ella', 68);
10 --Step 5: check the table
11 SELECT * FROM students.default.employees;
12 --question 4: A list of student scores. Convert numeric scores into letter grades
13 SELECT
14     student_name,
15     score,
16     CASE
17         WHEN score >= 90 THEN 'A'
18         WHEN score BETWEEN 80 AND 89 THEN 'B'
19         WHEN score BETWEEN 70 AND 79 THEN 'C'
20         WHEN score BETWEEN 60 AND 69 THEN 'D'
21         WHEN score < 60 THEN 'F'
22     END AS grade
23 FROM students.default.employees;

```

Add parameter

Table  

    

	student_name	score	grade
1	Anna	92	A
2	Ben	76	C
3	Cara	59	F
4	David	83	B
5	Ella	68	D

Question 5

```
1  --step 1: Create Catalog
2  CREATE CATALOG IF NOT EXISTS deliveries;
3  --Step 2: Create schema
4  CREATE SCHEMA IF NOT EXISTS deliveries.default;
5  --Step 3: Create table
6  CREATE TABLE IF NOT EXISTS deliveries.default.employees (delivery_id INT, delivery_time_minutes INT
7  );
8  --Step 4: Insert data into the table
9  INSERT INTO deliveries.default.employees VALUES (1, 45), (2, 80), (3, 30), (4, 65), (5, 100);
10 --Step 5: check the table
11 SELECT * FROM deliveries.default.employees;
```

Add parameter

Table ▼ + Q Y ⌵

	¹ ₃ delivery_id	¹ ₃ delivery_time_minutes
1	1	45
2	2	80
3	3	30
4	4	65
5	5	100

```

6 CREATE TABLE IF NOT EXISTS deliveries.default.employees (delivery_id INT, delivery_time_minutes INT
7 );
8 --Step 4: Insert data into the table
9 INSERT INTO deliveries.default.employees VALUES (1, 45), (2, 80), (3, 30), (4, 65), (5, 100);
10 --Step 5: check the table
11 SELECT * FROM deliveries.default.employees;
12 --Question 5:Label delivery performance based on the time taken.
13 SELECT
14 delivery_id,
15 delivery_time_minutes,
16 CASE
17 WHEN delivery_time_minutes <= 30 THEN 'fast'
18 WHEN delivery_time_minutes between 31 AND 60 THEN 'ON Time'
19 WHEN delivery_time_minutes > 60 THEN 'Late'
20 END AS performance
21 FROM deliveries.default.employees;

```

Add parameter

	delivery_id	delivery_time_minutes	performance
1	1	45	ON Time
2	2	80	Late
3	3	30	fast
4	4	65	Late
5	5	100	Late

Question 6

```
1  --step 1: Create a Catalog
2  CREATE CATALOG IF NOT EXISTS tickets
3  --step 2: Create schema
4  CREATE SCHEMA IF NOT EXISTS tickets.default
5  --step 3: Create table
6  CREATE TABLE IF NOT EXISTS tickets.default.employees (ticket_id INT, issue_type STRING, priority INT
7  );
8  --Step 4: Insert data
9  INSERT INTO tickets.default.employees VALUES (1, 'Login issues', 3), (2, 'Server down', 3), (3, 'Slow system', 2), (4, 'Email error', 2), (5, 'Password reset',1);
10 --Step 5 check the table
11 SELECT * FROM tickets.default.employees
```

Add parameter

Table	+	Q	Y	I	O	E	^	X
	ticket_id	issue_type	priority					
1	1	Login issues	3					
2	2	Server down	3					
3	3	Slow system	2					
4	4	Email error	2					
5	5	Password reset	1					

```
8  --Step 4: Insert data
9  INSERT INTO tickets.default.employees VALUES (1, 'Login issues', 3), (2, 'Server down', 3), (3, 'Slow system', 2), (4, 'Email error', 2), (5, 'Password reset'
10 --Step 5 check the table
11 SELECT * FROM tickets.default.employees
12 --Question 6: Support tickets stored with a numeric priority. Convert the priority into a readable label.
13
14 SELECT
15 issue_type,
16 priority,
17 CASE
18 WHEN priority = 3 THEN 'high'
19 WHEN priority = 2 THEN 'Medium'
20 WHEN priority = 1 THEN 'low'
21 END AS priority_label
22 FROM tickets.default.employees
```

Add parameter

Table	+	Q	Y	I	O	E	^	X
	issue_type	priority	priority_label					
1	Login issues	3	high					
2	Server down	3	high					
3	Slow system	2	Medium					
4	Email error	2	Medium					
5	Password reset	1	low					

Question 07: The attendance Table

Calculate each student's attendance percentage and classify the result.

```
1  -- Step 1: CREATE A Catalog
2  CREATE CATALOG IF NOT EXISTS exercise3;
3
4  --Step 2: Create a schema
5  CREATE SCHEMA IF NOT EXISTS exercise3.class;
6
7  -- STEP 3: CREATE a table
8  CREATE TABLE IF NOT EXISTS exercise3.class.attendance_brightlearn (
9      student_id INT,
10     days_present INT,
11     total_days INT
12 );
13
14 --STEP 4: Insert the data into the table
15 INSERT INTO TABLE exercise3.class.attendance_brightlearn VALUES
16 (1,45,50),
```

Add parameter

Table  

	^{1 2} ₃ student_id	^{1 2} ₃ days_present	^{1 2} ₃ total_days
1	1	45	50
2	2	30	50
3	3	48	50
4	4	25	50
5	5	50	50

```

21
22 --STEP 5: Check the table
23 SELECT*
24 FROM exercise3.class.attendance_brightlearn;
25 --Question 7:Calculate each student's attendance percentage and classify the result
26 SELECT
27     student_id,
28     (days_present/total_days)*100 AS attendance_percentage,
29     CASE
30         WHEN attendance_percentage>=90 THEN 'Excellent'
31         WHEN attendance_percentage BETWEEN 75 AND 89 THEN 'Good'
32         WHEN attendance_percentage<75 THEN 'Needs Improvement'
33     END AS attendance_status
34 FROM exercise3.class.attedance_brightlearn;
35
36

```

Add parameter

Table  

	¹ ₃ student_id	¹ ₃ days_present	¹ ₃ total_days
1	1	45	50
2	2	30	50
3	3	48	50
4	4	25	50
5	5	50	50

Question 8

```

1  --Step 1: Create the Catalog
2  CREATE CATALOG IF NOT EXISTS employees;
3  --Step 2: Create the schema
4  CREATE SCHEMA IF NOT EXISTS employees.default;
5  --Step 3: Create the Table
6  CREATE TABLE IF NOT EXISTS employees.default.products_inventory (
7      product_id INT,
8      stock_qty INT
9  );
10
11 --Step 4: Insert data into table
12 INSERT INTO employees.default.products_inventory VALUES
13 (1, 5), (2, 0), (3, 25), (4, 10), (5, 3);
14 --step 5: check the table
15 SELECT * FROM employees.default.products_inventory;

```

Add parameter

Table ▼ +

	¹ ₃ product_id	¹ ₃ stock_qty
1	1	5
2	2	0
3	3	25
4	4	10
5	5	3

```

10
11 --Step 4: Insert data into table
12 INSERT INTO employees.default.products_inventory VALUES
13 (1, 5), (2, 0), (3, 25), (4, 10), (5, 3);
14 --step 5: check the table
15 SELECT * FROM employees.default.products_inventory;
16 --Question 8: Stock levels per product. Label each row with the correct stock status.
17 SELECT
18 product_id,
19 stock_qty,
20 CASE
21 WHEN stock_qty = 0 THEN 'Out of stock'
22 WHEN stock_qty BETWEEN 1 AND 5 THEN 'Low stock'
23 WHEN stock_qty > 5 THEN 'In stock'
24 END AS stock_status
25 FROM employees.default.products_inventory;

```

Add parameter

Table  

	¹ ₃ product_id	¹ ₃ stock_qty	^A ₀ stock_status
1	1	5	Low stock
2	2	0	Out of stock
3	3	25	In stock
4	4	10	In stock
5	5	3	Low stock

Question 9

```
1  --Step 1: Create Catalog
2  CREATE CATALOG IF NOT EXISTS classes;
3  --step 2: Create schema
4  CREATE SCHEMA IF NOT EXISTS classes.default;
5  --step 3: Create the table
6  CREATE TABLE IF NOT EXISTS employees.default.classes (classes_id INT, subject STRING, enrolled_students INT
7  );
8  --step 4: Insert data into the table
9  INSERT INTO employees.default.classes VALUES
10 (1, 'Math', 30),
11 (2, 'English', 25),
12 (3, 'Science', 15),
13 (4, 'Art', 5),
14 (5, 'History', 20);
15
16 --step 5: check the table
```

Add parameter

Table  

	^{1 2} ₃ classes_id	^{A B} _C subject	^{1 2} ₃ enrolled_students
1	1	Math	30
2	2	English	25
3	3	Science	15
4	4	Art	5
5	5	History	20

```
12 (3, 'Science', 15),
13 (4, 'Art', 5),
14 (5, 'History', 20);
15
16 --step 5: check the table
17 SELECT * FROM employees.default.classes;
18 --question 9:Classes with their enrolled student counts. Classify each class by size.
19 SELECT
20 subject,
21 enrolled_students,
22 CASE
23 WHEN enrolled_students >= 25 THEN 'Large'
24 WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'
25 WHEN enrolled_students < 10 THEN 'Small'
26 END AS class_size_category
27 FROM employees.default.classes;
```

Add parameter

	^A _C subject	¹ ₃ enrolled_students	^A _C class_size_category
1	Math	30	Large
2	English	25	Large
3	Science	15	Medium
4	Art	5	Small
5	History	20	Medium

Question 10

```

1  --Step 1: Create the Catalog
2  CREATE CATALOG IF NOT EXISTS employees;
3  --Step 2: Create the Database
4  CREATE DATABASE IF NOT EXISTS employees.default;
5  --Step 3: Create the Table
6  CREATE TABLE IF NOT EXISTS employees.default.payments (
7      payment_id INT,
8      amount decimal,
9      payment_method STRING
10 );
11 --Step 4: Insert data into the table
12 INSERT INTO employees.default.payments
13 VALUES (1,50.00, 'Card'), (2,200.00, 'Cash'), (3,150.00, 'Card'), (4,75.00, 'paypal'),(5,300.00, 'cash');
14 --Step 5: check the table
15 SELECT * FROM employees.default.payments;
16 | --question 10: Combine the payment method and amount to decide whether a discount applies.

```

Add parameter

Table  

	¹ ₃ payment_id	.00 amount	^A _C payment_method
1	1	50	Card
2	2	200	Cash
3	3	150	Card
4	4	75	paypal
5	5	300	cash

```

10 );
11 --Step 4: Insert data into the table
12 INSERT INTO employees.default.payments
13 VALUES (1,50.00, 'Card'), (2,200.00, 'Cash'), (3,150.00, 'Card'), (4,75.00, 'paypal'),(5,300.00, 'cash');
14 --Step 5: check the table
15 SELECT * FROM employees.default.payments;
16 | --question 10: Combine the payment method and amount to decide whether a discount applies.
17 SELECT
18     payment_id,
19     payment_method,
20     amount,
21     CASE
22         WHEN payment_method = 'cash' AND amount >= 200 THEN 'Eligible for Discount'
23         ELSE 'Not Eligible'
24     END AS discount_eligibility
25 FROM employees.default.payments;

```

Add parameter

Table  

	¹ ₃ payment_id	^A _C payment_method	.00 amount	^A _C discount_eligibility
1	1	Card	50	Not Eligible
2	2	Cash	200	Not Eligible
3	3	Card	150	Not Eligible
4	4	paypal	75	Not Eligible
5	5	cash	300	Eligible for Discount

